7. Keylime 데이터베이스 스키마 및 데이터 모델 분석

7.1 개요

Keylime은 SQLAlchemy ORM을 사용하여 에이전트 상태, 정책, 그리고 검증 메타데이터를 관리합니다. 데이터베이스 스키마는 크게 두 부분으로 나뉩니다:

- **Verifier 데이터베이스**: 에이전트 검증 상태 및 정책 관리
- **Registrar 데이터베이스**: TPM 키 등록 및 인증서 관리

7.2 Verifier 데이터베이스 스키마 (`keylime/db/verifier_db.py`)

7.2.1 VerfierMain 테이블 구조

`VerfierMain` 클래스는 Keylime 시스템의 핵심 테이블로, 각 에이전트의 검증 상태와 메타데이터를 저장합니다:

```
class VerfierMain(Base):

__tablename__ = "verifiermain"

# 기본 식별자

agent_id = Column(String(80), primary_key=True)

# 네트워크 정보

ip = Column(String(15))

port = Column(Integer)
```

```
verifier_id = Column(String(80))

verifier_ip = Column(String(15))

verifier_port = Column(Integer)

# 상태 관리

operational_state = Column(Integer)

v = Column(String(45), nullable=True)

# TPM 및 암호화 관련

public_key = Column(String(500))

ak_tpm = Column(String(500))

revocation_key = Column(String(2800))

mtls_cert = Column(String(2048), nullable=True)

# 정책 관리 (외래키 관계)

tpm_policy = Column(JSONPickleType(pickler=JSONPickler))

ima_policy_id = Column(Integer, ForeignKey("allowlists.id"))

mb_policy_id = Column(Integer, ForeignKey("mbpolicies.id"))

# 알고리즘 지원

accept_tpm_hash_algs = Column(JSONPickleType(pickler=JSONPickler))

accept_tpm_encryption_algs = Column(JSONPickleType(pickler=JSONPickler))

accept_tpm_signing_algs = Column(JSONPickleType(pickler=JSONPickler))

hash_alg = Column(String(10))

enc_alg = Column(String(10))

sign_alg = Column(String(10))

# IMA 관련
```

```
ima_sign_verification_keys = Column(Text().with_variant(Text(429400000), "mysql"

ima_pcrs = Column(JSONPickleType(pickler=JSONPickler))

next_ima_ml_entry = Column(Integer)

learned_ima_keyrings = Column(JSONPickleType(pickler=JSONPickler))

# 부팅 및 시간 정보

boottime = Column(Integer)

tmp_clockinfo = Column(JSONPickleType(pickler=JSONPickler))

# PCR 데이터

pcr10 = Column(LargeBinary)

# 검증 통계

attestation_count = Column(Integer)

last_received_quote = Column(Integer)

last_successful_attestation = Column(Integer)

# 메타데이터 및 로깅

meta_data = Column(Text().with_variant(Text(429400000), "mysql"))

severity_level = Column(Integer, nullable=True)

last_event_id = Column(String(200), nullable=True)

supported_version = Column(String(20))
```

7.2.2 관계형 테이블 구조

VerifierAllowlist 테이블

IMA 정책을 관리하는 허용 목록 테이블:

```
class VerifierAllowlist(Base):

__tablename__ = "allowlists"

__table_args__ = (schema.UniqueConstraint("name", name="uniq_allowlists0name"),)

id = Column(Integer, primary_key=True)

name = Column(String(255), nullable=False)

checksum = Column(String(128))

generator = Column(Integer)

tpm_policy = Column(Text())

ima_policy = Column(Text().with_variant(Text(429400000), "mysql"))

# VerfierMain과의 관계

agent = relationship("VerfierMain", back_populates="ima_policy")
```

VerifierMbpolicy 테이블

Measured Boot 정책을 관리하는 테이블:

```
class VerifierMbpolicy(Base):

__tablename__ = "mbpolicies"

__table_args__ = (schema.UniqueConstraint("name", name="uniq_mbpolicies_name"),)

id = Column(Integer, primary_key=True)

name = Column(String(255), nullable=False)

mb_policy = Column(Text().with_variant(Text(429400000), "mysql"))

# VerfierMain과의 관계
```

```
agent = relationship("VerfierMain", back_populates="mb_policy")
```

7.2.3 테이블 간 관계 정의

VerfierMain 테이블은 다음과 같은 관계를 가집니다:

```
# VerfierMain 클래스 내부
ima_policy = relationship("VerifierAllowlist", back_populates="agent", uselist=F
mb_policy = relationship("VerifierMbpolicy", back_populates="agent", uselist=Fal
```

관계 특징:

- **일대일 관계**: 각 에이전트는 하나의 IMA 정책과 하나의 MB 정책을 가집니다
- **외래키 제약**: `ima_policy_id` 와 `mb_policy_id` 를 통한 참조 무결성
- **양방향 관계**: `back_populates` 를 통한 양방향 탐색 지원

7.3 Registrar 데이터베이스 스키마 (`keylime/db/registrar_db.py`)

7.3.1 RegistrarMain 테이블 구조

TPM 키 등록 및 인증서 관리를 위한 테이블:

```
class RegistrarMain(Base):
__tablename__ = "registrarmain"
# 기본 식별자
agent_id = Column(String(80), primary_key=True)
```

```
key = Column(String(45))

# TPM 키 관리

aik_tpm = Column(String(500)) # Attestation Identity Key

ek_tpm = Column(String(500)) # Endorsement Key

iak_tpm = Column(String(500)) # Initial Attestation Key

idevid_tpm = Column(String(500)) # Initial Device ID

# 인증서 관리

ekcert = Column(String(2048)) # EK Certificate

iak_cert = Column(String(2048)) # IAK Certificate

idevid_cert = Column(String(2048)) # IDevID Certificate

mtls_cert = Column(String(2048), nullable=True) # mTLS Certificate

# 네트워크 및 상태

ip = Column(String(15), nullable=True)

port = Column(Integer, nullable=True)

virtual = Column(Integer) # 가상 TPM 여부

active = Column(Integer) # 활성 상태

# 메타데이터

provider_keys = Column(JSONPickleType(pickler=JSONPickler))

regcount = Column(Integer) # 등록 횟수
```

7.4 JSONPickleType을 사용한 복합 데이터 저장

7.4.1 JSONPickleType 구현

Keylime은 복합 데이터 구조를 데이터베이스에 저장하기 위해 커스텀 `JSONPickleType` 을 사용합니다:

```
class JSONPickleType(PickleType):

impl = Text

cache_ok = True
```

7.4.2 JSONPickler 클래스 (`keylime/json.py`)

SQLAlchemy 요구사항을 충족하는 JSON 직렬화 래퍼:

```
class JSONPickler:

"""JSON pickler wrapper around json lib since SQLAlchemy invokes

dumps with extra positional parameters"""


@classmethod

def  dumps(cls, value: JSONType, *_args: Any, **_kwargs: Any) -> str:

"""Dumps the python value into a JSON string"""

return dumps(value)


@classmethod

def  loads(cls, value: Union[str, bytes]) -> Any:

"""Parses the JSON string and returns the corresponding python value"""

return loads(value)
```

7.4.3 복합 데이터 저장 사례

다음 필드들이 JSONPickleType을 사용하여 복합 데이터를 저장합니다:

1. **TPM 정책**: `tpm_policy` - TPM 검증 정책 객체
2. **알고리즘 지원**: `accept_tpm_hash_algs`, `accept_tpm_encryption_algs`, `accept_tpm_signing_algs`
3. **IMA PCR 데이터**: `ima_pcrs` - PCR 번호별 해시 값 딕셔너리
4. **학습된 키링**: `learned_ima_keyrings` - IMA 키링 정보
5. **TPM 클록 정보**: `tpm_clockinfo` - TPM 클록 상태
6. **제공자 키**: `provider_keys` - 클라우드 제공자별 키 정보

7.5 에이전트 상태 추적 및 메타데이터 관리

7.5.1 AgentAttestState 클래스 (`keylime/agentstates.py`)

메모리 내 에이전트 상태 관리를 위한 클래스:

```
class AgentAttestState:
"""AgentAttestState is used to support incremental attestation"""
agent_id: str
next_ima_ml_entry: int # 다음 IMA 측정 목록 엔트리
boottime: int # 부팅 시간
tpm_clockinfo: TPMClockInfo # TPM 클록 정보
tmp_state: TPMState # TPM PCR 상태
ima_pcrs: Set[int] # IMA가 사용하는 PCR 번호들
ima_keyring: ImaKeyrings # IMA 키링 객체
```

```
ima_dm_state: Optional[bytes] # Device Mapper 상태

quote_progress: Optional[Tuple[int, int]] # 인용 진행 상황
```

7.5.2 TPMState 클래스

TPM PCR 상태를 모델링하는 클래스:

```
class TPMState:

"""TPMState models the state of the TPM's PCRs"""

pcrs: Dict[int, Optional[bytes]] # PCR 번호별 해시 값

hash_alg: Dict[int, Hash] # PCR별 해시 알고리즘

def  init_pcr(self, pcr_num: int, hash_alg: Hash) -> None:

"""PCR 초기화"""

if pcr_num not in self.hash_alg:

self.hash_alg[pcr_num] = hash_alg

if  self.pcrs[pcr_num] is None:

self.pcrs[pcr_num] = TPMState.initial_pcr_value(pcr_num, hash_alg)
```

7.5.3 TPMClockInfo 데이터 클래스

TPM 클록 정보를 저장하는 데이터 클래스:

```
@dataclass

class TPMClockInfo:

clock: int # 현재 클록 값
```

```
resetcount: int # 리셋 횟수

restartcount: int # 재시작 횟수

safe: int # 안전 플래그

@classmethod

def  from_dict(cls, dclki: Dict[str, int]) -> "TPMClockInfo":

"""딕셔너리에서 TPMClockInfo 객체 생성"""

return  cls(

clock=dclki.get("clock", 0),

resetcount=dclki.get("resetCount", 0),

restartcount=dclki.get("restartCount", 0),

safe=dclki.get("safe", 1),

)
```

7.5.4 AgentAttestStates 싱글톤 관리자

전체 에이전트 상태를 관리하는 싱글톤 클래스:

```
class AgentAttestStates:

"""AgentAttestStates administers a map of AgentAttestState's indexed by agent_id

instance = None

map_lock: threading.Lock

map: Dict[str, AgentAttestState]

@staticmethod

def  get_instance() -> "AgentAttestStates":
```

```
"""싱글톤 AgentAttestState 생성 및 반환"""
if not AgentAttestStates.instance:
AgentAttestStates.instance = AgentAttestStates()
return AgentAttestStates.instance
```

7.6 데이터베이스 운영 특징

7.6.1 MySQL 호환성

대용량 텍스트 필드에 대한 MySQL 특별 처리:

```
meta_data = Column(Text().with_variant(Text(429400000), "mysql"))
ima_policy = Column(Text().with_variant(Text(429400000), "mysql"))
```

7.6.2 유니크 제약 조건

테이블별 고유성 보장:

```
# VerifierAllowlist
__table_args__ = (schema.UniqueConstraint("name", name="uniq_allowlists0name"),)

# VerifierMbpolicy
__table_args__ = (schema.UniqueConstraint("name", name="uniq_mbpolicies_name"),)
```

7.6.3 스레드 안전성

메모리 내 상태 관리에서 스레드 안전성 보장:

```
def get_by_agent_id(self, agent_id: str) -> AgentAttestState:
"""에이전트 ID로 상태 조회 (스레드 안전)"""
with self.map_lock:
agent_attest_state = self.map.get(agent_id)
if not agent_attest_state:
agent_attest_state = AgentAttestState(agent_id)
self.map[agent_id] = agent_attest_state
return agent_attest_state
```

7.7 요약

Keylime의 데이터베이스 스키마는 다음과 같은 특징을 가집니다:

1. **분리된 책임**: Verifier와 Registrar 데이터베이스의 명확한 역할 분담
2. **유연한 데이터 저장**: JSONPickleType을 통한 복합 객체 직렬화
3. **관계형 설계**: 외래키와 관계를 통한 데이터 무결성 보장
4. **확장성**: MySQL 대용량 텍스트 지원 및 인덱스 최적화
5. **메모리 캐싱**: 빈번한 접근을 위한 메모리 내 상태 관리
6. **스레드 안전성**: 동시성 환경에서의 안전한 상태 관리

이러한 설계를 통해 Keylime은 대규모 환경에서도 효율적인 TPM 기반 검증 시스템을 제공할 수 있습니다.